SOFTWARE REQUIREMENTS SPECIFICATION

For

Distributed Cloud Compute Network for Dynamic Resource Allocation

06 October 2026

Prepared by

Specialization	SAP ID	Name
CCVT	500123445	Govind Choudhary
CCVT	500124582	Neetesh Bisht
CCVT	500119667	Yash Khanna

School of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES
DEHRADUN - 248007, Uttarakhand

TABLE OF CONTENTS

No.	Topic	Page
	Revision History	3
1	INTRODUCTION	3
	1.1 Purpose of the Project	3
	1.2 Target Beneficiary	3
	1.3 Project Scope	3
	1.4 References	3
2	PROJECT DESCRIPTION	4
	2.1 Reference Algorithm	4
	2.2 Characteristic of Data	4
	2.3 SWOT Analysis	4
	2.4 Project Features	5
	2.5 User Classes and Characteristics	5
	2.6 Design and Implementation Constraints	5
	2.7 Design Diagrams	6-8
	2.8 Assumptions and Dependencies	8
3	SYSTEM REQUIREMENTS	9
	3.1 User Interface	9
	3.2 Software Interface	9
	3.3 Database Interface	9
	3.4 Protocols	9
4	NON-FUNCTIONAL REQUIREMENTS	10
	4.1 Performance Requirements	10
	4.2 Security Requirements	10
	4.3 Software Quality Attributes	10
5	OTHER REQUIREMENTS	11
	Appendix A: Glossary	11
	Appendix B: Analysis Model	11
	Appendix C: Issues List	11

REVISION HISTORY

Date	Change	Reason for Changes	Mentor Signature
06/10/2026	SRS v1.0	Baseline requirements drafted from working prototype and project synopsis	

1 INTRODUCTION

1.1 Purpose of the Project

Distributed Cloud Compute Network (DCCN) is a student-scale distributed computing platform that combines idle CPU, memory and optional GPU capacity from heterogeneous machines and exposes that capacity through a single control plane. The project addresses a practical mismatch: capable laboratory and workstation hardware is frequently under-used, while short-lived machine-learning, simulation and data-processing jobs may need more resources than one user machine can provide.

The current prototype focuses on resource registration, live telemetry, authenticated job submission, eligibility filtering, weighted node ranking, allocation records and auditable scheduling. It is intentionally smaller than a commercial cloud: the objective is to demonstrate transparent, resource-aware placement over intermittently available nodes using a clear API and role-specific portals.

1.2 Target Beneficiary

Primary beneficiaries are students, research groups and laboratories that need burst compute capacity without maintaining a dedicated cluster. The design follows the cloud principle of pooling configurable compute resources and making them available over a network [1]. Compute providers gain a controlled way to contribute machines and observe utilization, while administrators receive centralized visibility into users, node approval, job state, scheduling decisions and audit events.

1.3 Project Scope

The deliverable includes a FastAPI control plane, web portals for client/provider/admin roles, a SQLAlchemy persistence layer, a Python node agent using psutil for cross-platform CPU, memory and disk telemetry [5], job and node APIs, and a seven-factor scheduler. The prototype supports CPU/RAM/disk/GPU requirements, node heartbeats, queued jobs, manual or bulk scheduling, cancellation and score breakdowns. Actual container execution is represented by execution state in the mid-evaluation build; Docker daemon integration and production-grade recovery remain future implementation work.

1.4 References

- [1] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” NIST Special Publication 800-145, National Institute of Standards and Technology, Sept. 2011. Available: https://doi.org/10.6028/NIST.SP.800-145
- [2] FastAPI, “OAuth2 with Password (and hashing), Bearer with JWT tokens,” FastAPI Documentation. Available: https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/ (accessed Oct. 7, 2026).
- [3] SQLAlchemy Authors, “SQLAlchemy 2.0 Documentation: Object Relational Mapper and Session,” SQLAlchemy Documentation. Available: https://docs.sqlalchemy.org/en/20/orm/ (accessed Oct. 7, 2026).
- [4] Docker Inc., “Docker Engine Security,” Docker Documentation. Available: https://docs.docker.com/engine/security/ (accessed Oct. 7, 2026).
- [5] G. Rodola, “psutil documentation: system and process utilities,” psutil Documentation. Available: https://psutil.readthedocs.io/stable/ (accessed Oct. 7, 2026).

2 PROJECT DESCRIPTION

2.1 Reference Algorithm

Scheduling follows a filter -> rank -> reserve workflow. First, nodes that are not ONLINE, have stale telemetry (>30 s), or cannot satisfy CPU, RAM, disk, GPU count or requested VRAM are removed. Eligible nodes are then ranked using the implemented weighted score:

$$\mathbf{Score(i,j) = 0.22C + 0.18M + 0.25G + 0.12L + 0.08N + 0.10R + 0.05P}$$

C is normalized free CPU, M is free memory, G is GPU suitability, L rewards lower utilization, N rewards lower RTT, R reflects heartbeat recency, and P estimates packing efficiency. For CPU-only work, non-GPU nodes receive a higher GPU-suitability value so scarce GPU machines can be preserved. The highest-scoring feasible node is selected and the score breakdown is stored with the allocation for explainability.

Stage	**Implemented rule**
Filter	ONLINE node + fresh snapshot + sufficient unreserved CPU/RAM/disk/GPU.
Rank	Compute seven normalized factors and weighted composite score.
Reserve	Create allocation and update job state within the scheduler service.
Audit	Record scheduling and administrative actions with timestamps/outcomes.

2.2 Characteristic of Data

DCCN does not require a fixed external dataset. Its primary data is operational: static node hardware, periodic resource snapshots, user/job records, job requirements, allocations and audit events. Real agents obtain CPU, RAM and disk values through psutil; the demo can generate realistic simulated telemetry and GPU profiles. Snapshot data is time-stamped and treated as invalid for placement when it becomes stale.

2.3 SWOT Analysis

Area	**Project-specific assessment**
Strengths	Reuses heterogeneous hardware; explainable scheduling; role separation; lightweight agent.
Weaknesses	Central control plane; prototype execution path is partly simulated; GPU telemetry is basic.
Opportunities	Campus compute pool, ML inference/training experiments, rendering and scheduling research.
Threats	Node churn, untrusted workloads, driver mismatch, network instability and data leakage.

2.4 Project Features

- Role-based authentication for compute client, compute provider and administrator.
- Node registration with approval state, hardware inventory and periodic heartbeat telemetry.
- Job submission with Docker image metadata, CPU/RAM/disk/GPU requirements, timeout and priority fields.
- Constraint-based node filtering followed by explainable seven-factor weighted ranking.
- Queued job listing, individual scheduling, admin schedule-all, cancellation and allocation tracking.
- Dedicated client, provider and administrator portals plus unified operations dashboard and interactive API documentation.

2.5 User Classes and Characteristics

User class	**Characteristics / permitted actions**
Compute Client	Submits jobs, declares resource needs, views own jobs, triggers scheduling and cancels non-terminal jobs.
Compute Provider	Registers machines, runs the node agent and submits telemetry for owned nodes.
Administrator	Approves/updates nodes, views system-wide state, runs bulk scheduling and reviews administrative/audit information.

2.6 Design and Implementation Constraints

- Backend is Python/FastAPI with asynchronous SQLAlchemy access; current demo can operate on SQLite while configuration is PostgreSQL-ready.
- Node agents are Python processes; real telemetry depends on psutil. GPU hardware can be represented by simulated profiles in the demo build.
- Scheduling requires a recent ResourceSnapshot; snapshots older than 30 seconds are rejected by the eligibility filter.
- The current agent authenticates using provider credentials and sends bearer-token protected REST requests.
- The browser portals are served as static HTML by FastAPI. CORS is permissive in the prototype and must be restricted before deployment.
- Container launch/isolation is a planned integration boundary; the mid-evaluation build does not yet execute arbitrary workloads on remote Docker daemons.

2.7 Design Diagrams

The following diagrams describe the implemented prototype boundary and its principal interactions. They are original project diagrams prepared from the supplied source structure.

2.7 DESIGN DIAGRAMS

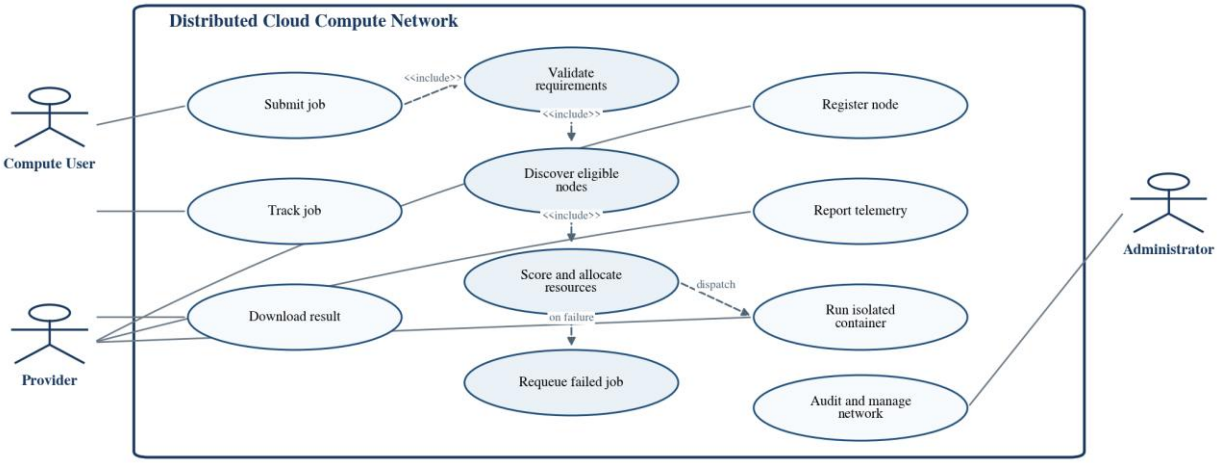


Figure 1. Level-2 use case diagram.

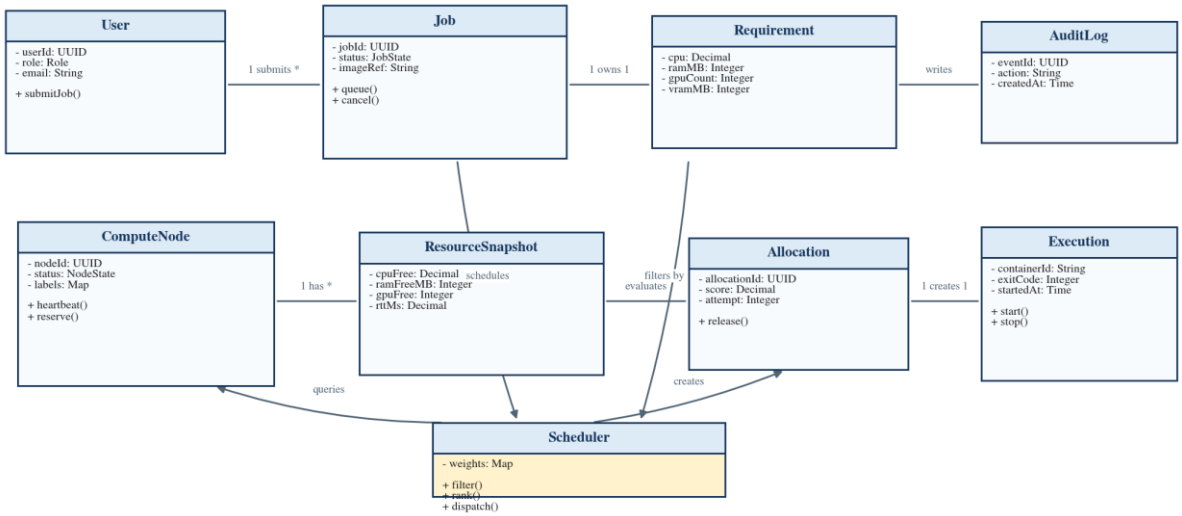


Figure 2. Core class diagram.

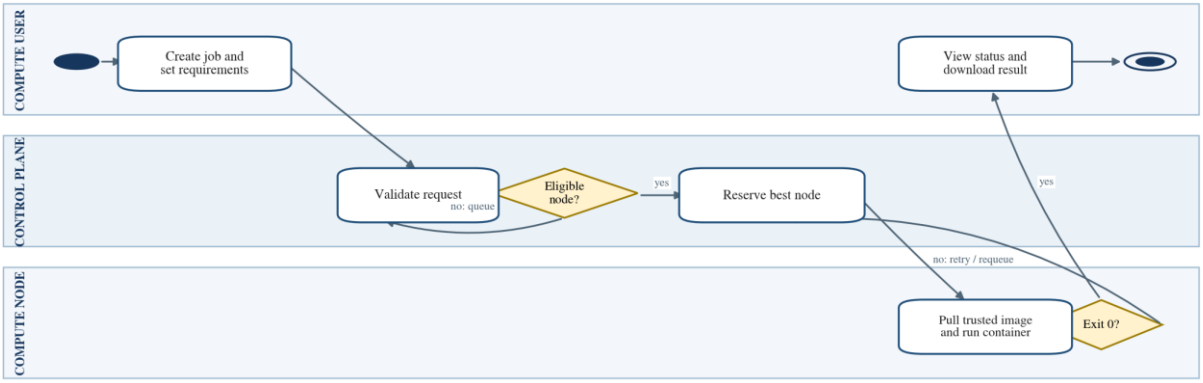


Figure 3. Job allocation activity diagram.

2.7 DESIGN DIAGRAMS (CONTINUED)

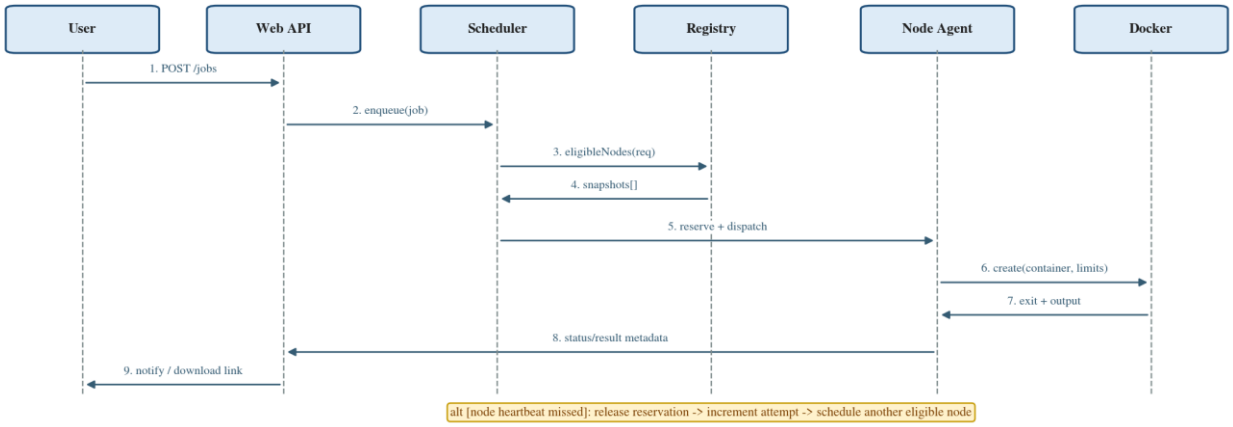


Figure 4. Successful job-attempt sequence diagram.

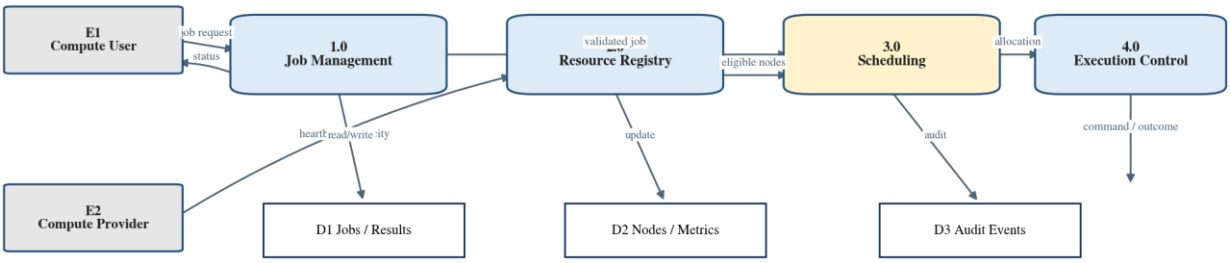


Figure 5. Level-1 data flow diagram.

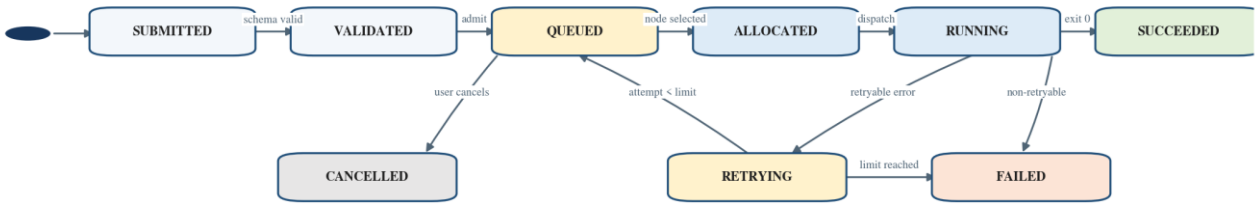


Figure 6. Job lifecycle state diagram.

2.7 DESIGN DIAGRAMS (CONTINUED)

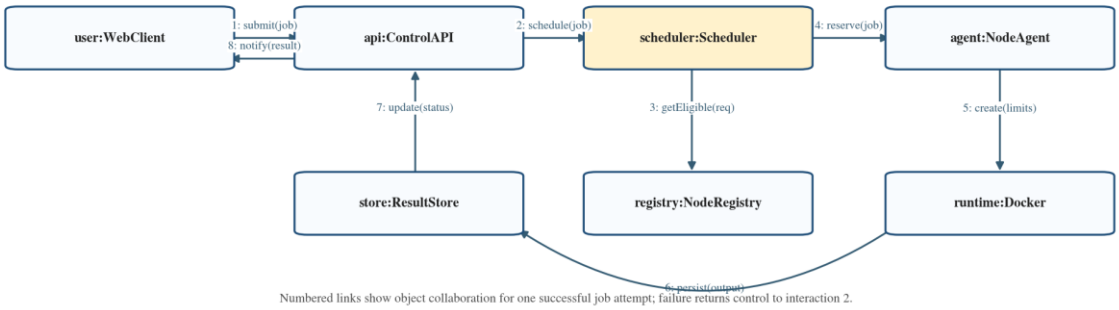


Figure 7. Object collaboration diagram.

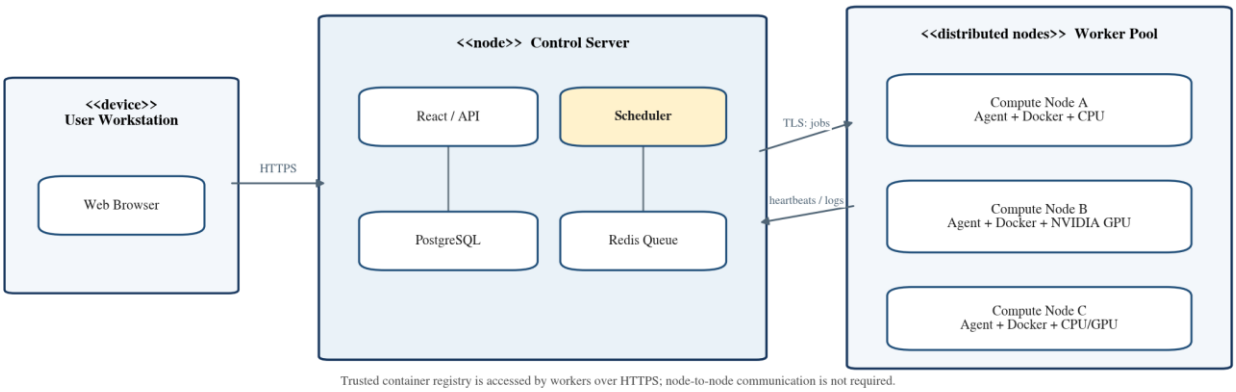


Figure 8. Prototype deployment diagram.

2.8 Assumptions and Dependencies

- Participants are known/trusted during prototype evaluation and provider credentials are available to node agents.
- Worker machines remain reachable often enough for heartbeats and scheduling; system clocks are reasonably synchronized.
- Python dependencies listed by server/agent requirements are installed; database initialization is completed before the demo.
- GPU availability in the demo may be simulated; real GPU execution will require compatible drivers and container runtime support.
- Failure recovery and remote container execution are treated as planned extensions where not present in the supplied implementation.

3 SYSTEM REQUIREMENTS

3.1 User Interface

The system shall provide a portal gateway and dedicated pages for compute clients, providers and administrators, plus a unified dashboard. Client screens shall support job submission and state visibility; provider screens shall expose node registration/telemetry context; admin screens shall support approval and operational control. The interface shall show explicit status labels and validation errors rather than relying only on color.

3.2 Software Interface

Interface	Requirement
Browser -> FastAPI	REST/JSON for API calls; static HTML portals served by the same application.
Agent -> Auth API	POST login request returns an access token used as Bearer authorization.
Agent -> Node API	Register hardware; POST heartbeat telemetry for the provider-owned node.
API -> Scheduler	Pass job identifier and AsyncSession to SchedulerService for placement.
Application -> DB	SQLAlchemy ORM persists users, nodes, snapshots, jobs, requirements, allocations and audit logs.

3.3 Database Interface

The persistence layer uses SQLAlchemy ORM models and session-based transaction handling [3]. The repository includes SQLite database files for local demonstration and configuration support for a server database. Referential links connect users to jobs/nodes, nodes to snapshots, and jobs to requirements/allocations. Allocation and scheduling state must be committed consistently so that a job is not silently assigned without corresponding evidence.

3.4 Protocols

- HTTP/REST with JSON request and response bodies for control-plane operations.
- Bearer-token authorization for protected routes; role dependencies enforce provider/admin operations.
- Heartbeat interval is configurable by the agent; scheduler freshness logic rejects snapshots older than 30 seconds.
- For production deployment, HTTPS/TLS is required and CORS must be limited to trusted origins.

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

Measure	Requirement / target
Scheduling decision	Filter and rank should complete interactively for a student-scale cluster; target <250 ms for ~100 candidate nodes excluding database/network delay.
API response	Ordinary metadata operations should normally complete within 500 ms on the local lab deployment.
Telemetry freshness	A snapshot older than 30 s must not be used for placement.
Heartbeat	Agent sends periodic telemetry; demo configuration should keep the interval short enough to maintain fresh state.
Scalability	Design should support growth from a few demo nodes to roughly 100 registered nodes without changing the scheduling model.

4.2 Security Requirements

- Passwords must be stored as secure hashes and authentication tokens must be validated before protected operations. FastAPI supports standards-based OAuth2 bearer authentication and signed JWT access tokens for this purpose [2].
- Role-based authorization must prevent ordinary clients from approving nodes or running administrator-only bulk actions.
- A provider may submit heartbeats only for nodes that belong to that provider.
- Production traffic must use TLS; secrets must be supplied through environment configuration and never committed to source control.
- CORS wildcard configuration is acceptable only for isolated demonstration and must be replaced with an allow-list in deployment.
- Future Docker execution must use approved images, non-root containers, resource limits and no workload access to the host Docker socket. Docker security guidance emphasizes namespace isolation, reduced Linux capabilities and strict control of daemon access [4].

4.3 Software Quality Attributes

Attribute	Required design response
Adaptability / Flexibility	Scheduler weights, API configuration and agent intervals should be externally configurable.
Availability / Reliability	Heartbeat-based liveness, durable state and clear terminal states limit ambiguity during node churn.
Correctness / Testability	Typed schemas, eligibility checks and deterministic scoring enable unit and integration tests.
Interoperability / Portability	REST/JSON and Python components run across common development hosts; container execution is OCI-oriented.
Maintainability / Reusability	Separate API, model, schema, scheduler, service and agent modules reduce coupling.
Robustness / Usability	Validation, explicit errors, role-specific portals and audit evidence make failures understandable.

5 OTHER REQUIREMENTS

The prototype should preserve an auditable record of significant scheduling and administrative actions. API documentation must remain available through the FastAPI documentation endpoint for testing. Configuration should be environment-driven, and demo seed data must be clearly distinguishable from real telemetry. Source control should exclude secrets, local databases and generated runtime artifacts where appropriate.

Appendix A: Glossary

Term	Meaning
Allocation	Record linking a job attempt to the compute node selected by the scheduler.
Control plane	Central API, persistence and scheduling logic coordinating the network.
Heartbeat	Periodic authenticated message carrying node liveness and telemetry.
ResourceSnapshot	Time-stamped CPU, RAM, disk, GPU and RTT measurements for a node.
VRAM	GPU video memory used as a hard requirement for GPU workloads.
Packing efficiency	Score factor that rewards useful residual capacity after placement.

Appendix B: Analysis Model

For a job j, the analysis first builds an eligible set E(j) containing only ONLINE nodes with a fresh snapshot and sufficient unreserved resources. Each eligible node receives the weighted score defined in Section 2.1. The highest score is selected. Functional verification should include: CPU-only placement, GPU-required placement, insufficient-resource rejection, stale-snapshot rejection, provider ownership on heartbeat, role restrictions, cancellation of active jobs and persistence of allocation/audit evidence.

Appendix C: Issues List

ID	Open issue	Status / planned resolution
I-01	Remote container execution	Planned: integrate agent with local Docker Engine using restricted execution policy.
I-02	Real GPU telemetry	Planned: add NVIDIA telemetry rather than demo-only GPU values.
I-03	Node-loss recovery	Planned: mark unavailable nodes and requeue retryable jobs.
I-04	Production CORS/TLS	Open: restrict origins and terminate TLS before non-lab deployment.
I-05	Persistent production DB	Open: validate PostgreSQL migration and concurrency behavior beyond SQLite demo.